MODUL PRAKTIKUM BIG DATA

MapReduce Sederhana dengan Hadoop dan Java

Hadoop 3.4.3 • HDFS • YARN • Java • Docker

Komponen	Lingkungan Praktikum
OS	Windows 10/11 + WSL2 + Docker Desktop
Hadoop	Apache Hadoop 3.4.3
Cluster	1 NameNode + 2 DataNode + 1 ResourceManager + 2 NodeManager
Program	Java MapReduce — WordCount

Petunjuk Aplikasi yang Digunakan

Setiap langkah diberi keterangan aplikasi agar mahasiswa jelas harus menulis file atau menjalankan perintah di mana.

- 🟦 JAVA / VS CODE : menulis Mapper, Reducer, dan Driver.
- 🟩 DOCKER COMPOSE / VS CODE : membuat konfigurasi dan service/container Hadoop.
- 🟧 POWERSHELL / CMD : menjalankan perintah Docker, HDFS, dan submit job.
- ⬛ TERMINAL DI DALAM CONTAINER : menjalankan javac, jar, dan pengecekan Linux/Hadoop.
- 🌐 BROWSER : membuka HDFS Web UI dan YARN Web UI.

1. Tujuan Praktikum

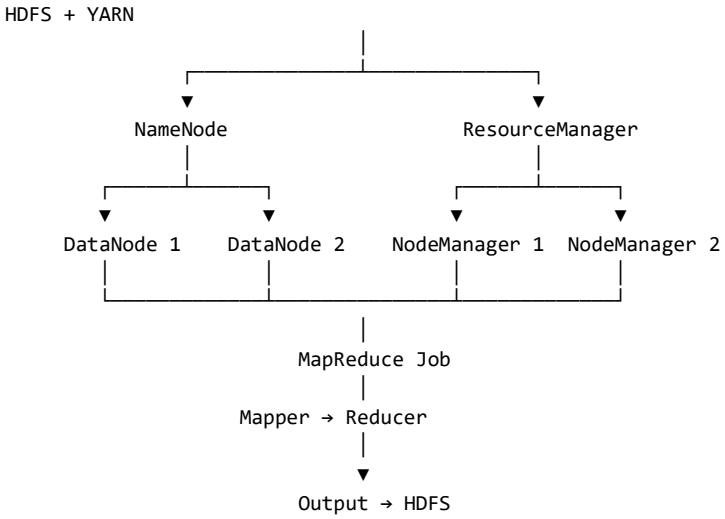
- Memahami paradigma MapReduce: Map → Shuffle & Sort → Reduce.
- Memahami hubungan HDFS, YARN, dan MapReduce.
- Membuat program WordCount menggunakan Java Hadoop.
- Menjalankan job pada cluster dengan dua DataNode dan dua NodeManager.
- Mengamati bahwa output MapReduce disimpan kembali di HDFS.

2. Prasyarat

Praktikum melanjutkan modul HDFS sebelumnya. Diasumsikan WSL2, Docker Desktop, image apache/hadoop:3.4.3, dan project HDFS sudah tersedia.

- Folder kerja contoh: C:\bigdata\mapreduce
- Folder konfigurasi: C:\bigdata\mapreduce\conf
- Source Java dapat disimpan pada subfolder mapreduce-wordcount.

3. Arsitektur Praktikum



- NameNode: mengelola metadata HDFS.
- DataNode: menyimpan block data HDFS.
- ResourceManager: mengelola resource dan scheduling YARN.
- NodeManager: menjalankan dan memantau container/task pada worker.
- Driver: mengonfigurasi dan mensubmit job MapReduce; Driver bukan YARN.
- Mapper dan Reducer: kode Java yang dijalankan sebagai task.

4. LANGKAH 1: Membangun Cluster HDFS + YARN

Bagian ini dilakukan sebelum pemrograman Java. File XML hanya mengatur konfigurasi; container/daemon YARN harus didefinisikan dan dijalankan melalui Docker Compose.

Konsep kunci: mapred-site.xml dan yarn-site.xml = konfigurasi. docker-compose.yml = mendefinisikan service. yarn resourcemanager dan yarn nodemanager = menjalankan daemon.

4.1 mapred-site.xml

🟩 **DOCKER COMPOSE / VS CODE — file konfigurasi**

```
<?xml version="1.0"?>
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>
```

4.2 yarn-site.xml

🟩 **DOCKER COMPOSE / VS CODE — file konfigurasi**

```
<?xml version="1.0"?>
<configuration>
    <property>
        <name>yarn.resourcemanager.hostname</name>
        <value>resourcemanager</value>
    </property>
    <property>
```

```
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>
    <property>
        <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
        <value>org.apache.hadoop.mapred.ShuffleHandler</value>
    </property>
    <property>
        <name>yarn.nodemanager.vmem-check-enabled</name>
        <value>false</value>
    </property>
</configuration>
```

4.3 docker-compose.yml

DOCKER COMPOSE / VS CODE — file docker-compose.yml

Tambahkan ResourceManager dan dua NodeManager ke cluster HDFS. Pada lab sederhana, worker dapat menjalankan DataNode dan NodeManager sekaligus.

```
services:

  namenode:
    image: apache/hadoop:3.4.3
    container_name: namenode
    hostname: namenode
    user: root
    command: >
      bash -c "
        mkdir -p /tmp/hadoop-data/namenode &&
        chown -R hadoop:hadoop /tmp/hadoop-data/namenode &&
        if [ ! -d /tmp/hadoop-data/namenode/current ]; then
          su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs namenode -format -force -nonInteractive';
        fi &&
        su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs namenode'
      "
    ports:
      - "9870:9870"
      - "8020:8020"
    volumes:
      - ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
      - ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
      - ./conf/mapred-site.xml:/opt/hadoop/etc/hadoop/mapred-site.xml:ro
      - ./conf/yarn-site.xml:/opt/hadoop/etc/hadoop/yarn-site.xml:ro
      - namenode-data:/tmp/hadoop-data

  datanode1:
    image: apache/hadoop:3.4.3
    container_name: datanode1
    hostname: datanode1
    user: root
    command: >
      bash -c "
        mkdir -p /tmp/hadoop-data/datanode &&
        chown -R hadoop:hadoop /tmp/hadoop-data/datanode &&
        su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs datanode'
      "
    volumes:
      - ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
      - ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
      - ./conf/mapred-site.xml:/opt/hadoop/etc/hadoop/mapred-site.xml:ro
      - ./conf/yarn-site.xml:/opt/hadoop/etc/hadoop/yarn-site.xml:ro
      - datanode1-data:/tmp/hadoop-data
```

```
  depends_on: [namenode]

datanode2:
  image: apache/hadoop:3.4.3
  container_name: datanode2
  hostname: datanode2
  user: root
  command: >
    bash -c "
      mkdir -p /tmp/hadoop-data/datanode &&
      chown -R hadoop:hadoop /tmp/hadoop-data/datanode &&
      su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs datanode'
    "
  volumes:
    - ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
    - ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
    - ./conf/mapred-site.xml:/opt/hadoop/etc/hadoop/mapred-site.xml:ro
    - ./conf/yarn-site.xml:/opt/hadoop/etc/hadoop/yarn-site.xml:ro
    - datanode2-data:/tmp/hadoop-data
  depends_on: [namenode]

resourcemanager:
  image: apache/hadoop:3.4.3
  container_name: resourcemanager
  hostname: resourcemanager
  user: root
  command: >
    bash -c "
      mkdir -p /tmp/hadoop-yarn &&
      chown -R hadoop:hadoop /tmp/hadoop-yarn &&
      su -s /bin/bash hadoop -c '/opt/hadoop/bin/yarn resourcemanager'
    "
  ports:
    - "8088:8088"
  volumes:
    - ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
    - ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
    - ./conf/mapred-site.xml:/opt/hadoop/etc/hadoop/mapred-site.xml:ro
    - ./conf/yarn-site.xml:/opt/hadoop/etc/hadoop/yarn-site.xml:ro
    - resourcemanager-data:/tmp/hadoop-yarn

nodemanager1:
  image: apache/hadoop:3.4.3
  container_name: nodemanager1
  hostname: nodemanager1
  user: root
  command: >
    bash -c "
      mkdir -p /tmp/hadoop-yarn &&
      chown -R hadoop:hadoop /tmp/hadoop-yarn &&
      su -s /bin/bash hadoop -c '/opt/hadoop/bin/yarn nodemanager'
    "
  ports:
    - "8042:8042"
  volumes:
    - ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
    - ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
    - ./conf/mapred-site.xml:/opt/hadoop/etc/hadoop/mapred-site.xml:ro
    - ./conf/yarn-site.xml:/opt/hadoop/etc/hadoop/yarn-site.xml:ro
    - nodemanager1-data:/tmp/hadoop-yarn
  depends_on: [resourcemanager, datanode1]

nodemanager2:
  image: apache/hadoop:3.4.3
```

```
    container_name: nodemanager2
    hostname: nodemanager2
    user: root
    command: >
      bash -c "
        mkdir -p /tmp/hadoop-yarn &&
        chown -R hadoop:hadoop /tmp/hadoop-yarn &&
        su -s /bin/bash hadoop -c '/opt/hadoop/bin/yarn nodemanager'
      "
    ports:
      - "8043:8042"
    volumes:
      - ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
      - ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
      - ./conf/mapred-site.xml:/opt/hadoop/etc/hadoop/mapred-site.xml:ro
      - ./conf/yarn-site.xml:/opt/hadoop/etc/hadoop/yarn-site.xml:ro
      - nodemanager2-data:/tmp/hadoop-yarn
    depends_on: [resourcemanager, datanode2]

volumes:
  namenode-data:
  datanode1-data:
  datanode2-data:
  resourcemanager-data:
  nodemanager1-data:
  nodemanager2-data:
```

Catatan: port host NodeManager 1 = 8042, NodeManager 2 = 8043. Keduanya tetap menggunakan port internal container 8042 sehingga tidak bentrok.

4.4 Menjalankan Cluster

🟧 **POWERSHELL / CMD — dari folder docker-compose.yml**

```
docker compose up -d
docker compose ps
```

Pastikan enam service utama terlihat running: NameNode, DataNode 1, DataNode 2, ResourceManager, NodeManager 1, dan NodeManager 2.

4.5 Verifikasi YARN

🟧 **POWERSHELL / CMD**

```
docker exec -it resourcemanager /opt/hadoop/bin/yarn node -list
```

Target: Total Nodes: 2.

4.6 Verifikasi HDFS

🟧 **POWERSHELL / CMD**

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfsadmin -report
```

Target: Live datanodes (2).

4.7 Verifikasi melalui Browser

🌐 **BROWSER**

HDFS NameNode UI: http://localhost:9870

YARN ResourceManager UI: http://localhost:8088

5. Menyiapkan Input di HDFS

🟧 **POWERSHELL / CMD**

```
docker cp .\input.txt namenode:/tmp/input.txt
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum/mapreduce/input
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -put -f /tmp/input.txt /praktikum/mapreduce/input/
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/mapreduce/input
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -cat /praktikum/mapreduce/input/input.txt
```

Contoh input.txt: (contoh input boleh diubah isinya)

```
big data
hadoop
big data
hadoop hdfs
big data hadoop
```

6. Paradigma MapReduce

```
INPUT
  ↓
MAP → (key, value)
  ↓
SHUFFLE & SORT → key dikelompokkan
  ↓
REDUCE → hasil agregasi
  ↓
OUTPUT → HDFS
```

7. Tipe Data Hadoop (Writable)

Java	Hadoop Writable
String	Text
int	IntWritable
long	LongWritable
float	FloatWritable
double	DoubleWritable
boolean	BooleanWritable

Mapper menggunakan Mapper<KEYIN, VALUEIN, KEYOUT, VALUEOUT>. WordCount: Mapper<Object, Text, Text, IntWritable>.

8. Implementasi Mapper

🟦 **JAVA / VS CODE: simpan file dengan nama WordCount.java**

```
public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one =
            new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context)
```

```
            throws IOException, InterruptedException {

        StringTokenizer itr =
                new StringTokenizer(value.toString());

        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

9. Implementasi Reducer

🟦 **JAVA / VS CODE : WordCount.java**

```
public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context)
            throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}
```

10. Implementasi Driver

🟦 **JAVA / VS CODE : WordCount.java**

```
public static void main(String[] args)
        throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");

    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setReducerClass(IntSumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

Driver mengatur konfigurasi dan submit job. Driver bukan ResourceManager dan bukan NodeManager.

11. Program Lengkap WordCount.java

JAVA / VS CODE

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
            extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {

            StringTokenizer itr =
                    new StringTokenizer(value.toString());

            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
                           Context context)
                throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }

            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
```

```
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

12. Kompilasi Java dan Membuat JAR

🟦 **JAVA / VS CODE : menulis WordCount.java**

🟧 **POWERSHELL / CMD : mengirim file ke container**

```
docker cp .\mapreduce-wordcount\WordCount.java namenode:/tmp/WordCount.java
docker exec -it namenode bash
```

⬛ **TERMINAL DI DALAM CONTAINER : javac dan jar**

```
java -version
javac -version
cd /tmp
mkdir -p classes

javac -cp
"/opt/hadoop/share/hadoop/common/*:/opt/hadoop/share/hadoop/common/lib/*:/opt/hadoop/share/hadoop/hdfs/*:/opt
/hadoop/share/hadoop/hdfs/lib/*:/opt/hadoop/share/hadoop/mapreduce/*:/opt/hadoop/share/hadoop/mapreduce/lib/*
:/opt/hadoop/share/hadoop/yarn/*:/opt/hadoop/share/hadoop/yarn/lib/*" -d classes WordCount.java

jar -cvf /tmp/wordcount.jar -C /tmp/classes/ .
jar tf /tmp/wordcount.jar
exit
```

13. Submit dan Menjalankan MapReduce Job

🟧 **POWERSHELL / CMD**

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -rm -r /praktikum/mapreduce/output

docker exec -it namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount /praktikum/mapreduce/input
/praktikum/mapreduce/output
```

```
PowerShell
   ↓
Driver (Java)
   ↓
YARN ResourceManager
   ↓
NodeManager 1 / NodeManager 2
   ↓
Mapper + Shuffle/Sort + Reducer
   ↓
Output → HDFS
```

14. Melihat Output

🟧 **POWERSHELL / CMD**

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/mapreduce/output
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -cat /praktikum/mapreduce/output/part-r-00000
```

Contoh hasil:

```
big     3
data    3
hadoop  3
hdfs    1
```

15. Mengamati Job melalui YARN

🌐 **BROWSER**

Buka http://localhost:8088. Amati aplikasi MapReduce pada YARN ResourceManager UI. Jika job sangat cepat selesai, riwayat aplikasi mungkin lebih mudah diamati melalui bagian history yang tersedia pada lingkungan Hadoop.

16. Bagaimana Output Disimpan?

Output MapReduce ditulis ke HDFS. NameNode mengelola metadata file dan lokasi block, sedangkan DataNode menyimpan block. YARN berfungsi mengatur resource dan eksekusi aplikasi, bukan sebagai penyimpanan permanen.

- NameNode = metadata
- DataNode = block data
- ResourceManager = resource/scheduling
- NodeManager = eksekusi container/task
- HDFS = storage
- YARN = resource management + execution framework

17. Troubleshooting

Gejala	Pengecekan
Total Nodes: 0	Cek ResourceManager dan log NodeManager.
Live datanodes (0)	Cek DataNode 1/2 dan fs.defaultFS.
Output directory already exists	Hapus folder output dengan hdfs dfs -rm -r.
Port bentrok	Pastikan NodeManager 2 memakai 8043:8042.
Job gagal	Periksa mount mapred-site.xml/yarn-site.xml dan log container.

Hindari docker compose down -v kecuali memang ingin menghapus named volume dan menginisialisasi ulang data cluster.

19. Tugas Praktikum

1. Buktikan terdapat 2 DataNode dengan hdfs dfsadmin -report.
2. Buktikan terdapat 2 NodeManager dengan yarn node -list.

3. Jalankan WordCount menggunakan input minimal 10 kata berbeda.
4. Jelaskan proses Map → Shuffle & Sort → Reduce pada data yang digunakan.
5. Ambil screenshot YARN UI dan output HDFS.
6. Jelaskan perbedaan NameNode, DataNode, ResourceManager, NodeManager, Mapper, Reducer, dan Driver.

21. Kesimpulan

Praktikum ini menggabungkan HDFS, YARN, dan MapReduce. HDFS menangani penyimpanan terdistribusi melalui NameNode dan DataNode. YARN mengelola resource melalui ResourceManager dan NodeManager. Program Java menyediakan Mapper, Reducer, dan Driver. Driver mensubmit job, YARN mengatur eksekusi task pada worker, dan hasil akhirnya disimpan pada HDFS.